

Predlog projekta

Pametni asistent za društvenu igru Wingspan - Big Bird Brain

Dušan Komadinović SV65/2022

Opis problema

Motivacija

Društvena igra *Wingspan* predstavlja takmičarsku stratešku *engine-building* igru za dva do pet igrača. U njoj igrači razvijaju sopstveni ekosistem ptica kroz pažljivo planiranje poteza i upravljanje resursima sa ciljem da prikupe što veći broj bodova. Partija traje četiri runde sa ukupno 26 poteza (8, 7, 6 i 5 poteza po rundi).

Efikasnost odluke ne zavisi samo od trenutne koristi. Strateška dubina igre proizilazi iz velikog broja međusobno zavisnih faktora koji utiču na vrednost svakog poteza, kao što su:

- već odigrane ptice i njihove sposobnosti,
- dostupni resursa hrane,
- sinergije (*synergy*) između staništa,
- ciljevi na kraju runde,
- preostali broj poteza u partiji i
- indirektni uticaj poteza protivnika.

Zbog ovakve strukture odlučivanja, početnici i igrači srednjeg nivoa imaju poteškoća da procene koja akcija donosi najveću dugoročnu vrednost. Moraju da analiziraju veliki broj varijabli i posledica budućih poteza, što nije lako ni za iskusnije igrače.

Ovakav domen predstavlja pogodan problem za primenu sistema baziranih na znanju, jer odluke koje donose iskusni igrači počivaju na skupu heurističkih pravila i ekspertskog iskustva, kao što su:

- rano razvijanje efikasnog *engine*-a,
- optimizacija resursa,
- prilagođavanje strategije ciljevima runde,
- balansiranje kratkoročne i dugoročne koristi.

Motivacija ovog projekta je razvoj **pametnog asistenta zasnovanog na znanju** koji formalizuje strateško razmišljanje igrača i omogućava njegovu primenu pri analizi konkretnih situacija tokom

partije. Cilj je kreiranje sistema koji može da pomogne igračima u razumevanju posledica svojih odluka i prepoznavanju kvalitetnih strateških izbora poteza koji možda nisu odmah očigledni. Ujedno može da posluži kao alat za učenje strategije i analize partija.

Napomena

Postoje razne ekspanzije za igru koje uvode nova pravila i mehanike igre. Fokus projekta je samo na osnovnoj igri.

Pregled problema

Tokom jednog poteza u *Wingspan*-u igrač bira između četiri osnovne akcije:

- igranje nove ptice u jedno od staništa (šuma, livada, močvara),
- uzimanje hrane,
- polaganje jaja i
- uzimanje novih karata ptica.

Iako je skup mogućih akcija ograničen, procena njihove stvarne vrednosti predstavlja složen problem odlučivanja. Optimalan potez zavisi od velikog broja međusobno povezanih faktora, uključujući:

- trenutno razvijen *engine*,
- dostupne resurse,
- ciljeve runde i
- preostali broj poteza.

Vrednost pojedinačne akcije često se ne može proceniti izolovano, već isključivo u kontekstu kompletnog stanja igre.

Problem koji se rešava ovim projektom može se formulisati na sledeći način:

Kako formalizovati ekspertske strateško znanje igrača Wingspan-a i iskoristiti ga za generisanje objašnjene preporuke optimalne akcije u datom stanju igre?

Postojeća rešenja

Postojeći digitalni alati vezani za Wingspan mogu se svrstati u dve osnovne kategorije:

1. Referentni alati:

- baze podataka o kartama,
- sajтови za strategije i preporuke,
- digitalni priručnici pravila i
- sajтови za vođenje rezultata.

Ovi alati služe kao pomoć pri učenju pravila ili evidenciji partije, ali ne pružaju stratešku analizu niti preporuke za donošenje odluka.

2. Digitalna adaptacija igre

Zvanična digitalna verzija igre omogućava igranje protiv računara ili drugih igrača, ali ne objašnjava razloge iza poteza niti modeluje proces strateškog razmišljanja.

Iako ovi alati olakšavaju igranje, nijedan od njih ne funkcioniše kao **inteligentni savetodavni sistem** koji analizira stanje partije i pomaže igraču da razume zašto je određeni potez dobar ili loš. **Ne postoji javno dostupno rešenje** zasnovano na pravilima koje prihvata strukturisano stanje igre i generiše rangiranu, obrazloženu listu preporučenih poteza.

Prednost predloženog rešenja

Predloženi sistem bi sadržao:

1. Sveobuhvatno rezonovanje

Sistem analizira celokupno stanje igre umesto izolovanih elemenata poput pojedinačnih karata ili trenutnog rezultata.

2. Rangirane i objašnjene preporuke

Sistem generiše listu mogućih akcija zajedno sa objašnjenjem procesa zaključivanja i aktiviranih pravila.

3. Transparentnu bazu znanja

Znanje je predstavljeno kroz eksplicitna pravila koja se mogu proveravati, menjati i proširivati, što sistem čini pogodnim za učenje i analizu strategije.

Metodologija rada

Predloženi sistem predstavlja **ekspertski savetodavni sistem** koji nad formalizovanim opisom trenutnog stanja igre generiše preporuke poteza korišćenjem baze znanja i mehanizama rezonovanja.

Ulazi u sistem (*Input*)

Ulaz u sistem predstavlja strukturisani opis trenutnog stanja partije igre. Ulazni podaci modeluju informacije koje su dostupne ljudskom igraču u realnoj partiji. Sistem prihvata sledeće informacije:

Stanje partije:

- trenutna runda igre,
- broj preostalih poteza u rundi,
- aktivni cilj runde i
- napredak igrača u odnosu na protivnike.

Tabla igrača:

- ptice postavljene po staništima:
 - šuma,
 - livada i
 - močvara
- broj jaja po ptici,
- keširana hrana ili karte na pticama i
- ukupna snaga svakog staništa (broj aktivacija efekata ptica).

Resursi igrača:

- dostupni tokeni hrane po tipu (crv, bobice, riba, miš, pšenica),
- karte ptica u ruci sa atributima:
 - zahtevi staništa,
 - cena hrane,
 - broj poena,

- tip sposobnosti i
- kategorija efekta.

Strateški kontekst:

- bonus karte igrača i
- vidljivo stanje protivnika (broj ptica po staništima i približan napredak).

Ulaz će se unositi preko korisničkog interfejsa, nakon čega se pretvara u skup činjenica u radnoj memoriji sistema.

Izlazi iz sistema (*Output*)

Na osnovu ulaznih činjenica sistem generiše **preporuku poteza**. Izlaz sistema obuhvata:

- rangiranu listu preporučenih akcija:
 - igranje ptice,
 - uzimanje hrane,
 - polaganje jaja i
 - uzimanje novih karata
- numerički prioritet svake akcije,
- objašnjenje rezonovanja (lanac aktiviranih pravila),
- sugestije napretka ka bonus ciljevima i
- strategijska upozorenja, kao što su:
 - zaostajanje u cilju runde,
 - nedostatak resursa i
 - neefikasan razvoj *engine*-a.

Primer izlaza:

Preporuka: Igrati pticu X u staništu livade

Prioritet: 8.7

Obrazloženje: poboljšava produkciju jaja, omogućava aktivaciju sinergije i povećava šansu ostvarivanja cilja runde.

Baza znanja

Baza znanja predstavlja centralni deo sistema i sastoji se iz statičkog i dinamičkog znanja.

Činjenice (*Facts*)

Statičko znanje

1. Baza podataka karata ptica (170 ptica osnovne igre), modelovana kroz sledeće attribute:

- staništa,
- cena hrane,
- broj poena,
- kapacitet gnezda,
- raspon krila i
- tip sposobnosti:
 - bez sposobnosti,
 - pri igranju,
 - aktivacija,
 - između poteza,
 - kraj runde;
- funkcionalna kategorija efekta:
 - generisanje hrane,
 - polaganje jaja,
 - vučenje karata,
 - čuvanje hrane i
 - *tuck* mehanike.

2. Baza podataka karata bonus ciljeva

Bonus karte predstavljaju skrivene dugoročne ciljeve, svaki igrač počinje sa jednim bonus ciljem koji samo on zna, a u toku partije može izvući nove.

Svaka bonus kartica treba da bude modelovana kao strukturisani skup uslova:

- naziv kartice,
- uslov bodovanja,
- atribut koji se proverava,
- način računanja poena.

Primeri:

- broj ptica određene kategorije,
- ptice sa određenim tipom gnezda,
- ptice određenog raspona krila,
- ptice u određenom staništu.

3. Baza *end-of-round* ciljeva

U svakoj rundi igrači se nadmeću da što bolje ispune cilj za tu rundu. Mogu se skupljati u *casual* i *competitive* režimu, pri čemu je drugi oštiji pri bodovanju.

Svaka pločica cilja definiše:

- tip cilja (broj ptica, broj jaja itd.),
- relevantno stanište ili atribut,
- način računanja rezultata i
- kriterijum poređenja između igrača.

Ove informacije omogućavaju sistemu da rezonuje o strategijskoj hitnosti i prilagođava preporuke u zavisnosti od aktivnog cilja runde.

Primeri:

- ukupan broj ptica u šumi,
- ukupan broj jaja u močvari,
- ukupan broj ptica,
- broj jaja u gnezdu sa određenim simbolom

Dinamičko znanje

Činjenice generisane iz korisničkog unosa:

- trenutno stanje table,
- raspoloživi resursi,
- stanje runde,
- napredak ka ciljevima i
- istorija poteza tokom partije.

Pravila (Rule Base)

Ekspertsko znanje biće formalizovano kroz skup heurističkih pravila.

Primeri pravila:

- ako igrač nema stabilan izvor hrane: povećati prioritet akcije uzimanja hrane,
- ako stanište šume sadrži najmanje dve ptice: igrač poseduje stabilan izvor hrane (*developed food habitat*) i
- ako se približava kraj runde: povećati težinu akcija koje doprinose cilju runde.

Forward Chaining rezonovanje

Sistem koristi *forward chaining* mehanizam zaključivanja.

Proces rezonovanja odvija se kroz više nivoa:

1. evaluacija resursa,
2. detekcija obrazaca igre (*engine inference*),
3. procena strategijske situacije,
4. generisanje kandidata poteza,
5. izbor preporuke.

Zaključci jednog nivoa postaju ulaz za sledeći nivo, čime se ostvaruje ulančavanje pravila.

Accumulate funkcija

Accumulate operator koristi se za agregaciju znanja:

- sabiranje ukupne vrednosti staništa,
- procenu produkcione moći *engine*-a,
- izračunavanje strategijskog skora akcija i
- procenu napretka ka ciljevima.

Na osnovu akumuliranih vrednosti generišu se rangirane preporuke.

Backward Chaining

Backward chaining se koristi za validaciju i objašnjenje odluka.

Primer:

„Da li je igranje ptice optimalna akcija?“

Sistem pokušava da dokaže cilj proverom potrebnih uslova:

- dostupni resursi,
- strategijska korist i
- doprinos ciljevima.

Na ovaj način omogućava se objašnjivo rezonovanje.

CEP - Complex Event Processing

Sistem prati tok partije kroz događaje:

- više uzastopnih poteza bez razvoja *engine*-a,
- približavanje kraju runde,
- nagle promene resursa i
- stagnacija u ostvarenju ciljeva.

Sekvence događaja generišu nove činjenice koje aktiviraju dodatna pravila rezonovanja.

CEP omogućava da sistem reaguje ne samo na trenutno stanje, već i na **evoluciju igre kroz tok partije**.

Evolucija baze znanja

Nakon svakog poteza:

- stanje igre se ažurira,
- nova činjenica se dodaje u radnu memoriju,
- istorija događaja se čuva i
- sistem može analizirati trendove tokom partije.

Interakcije zasnovane na znanju

Proces rada sistema zasniva se na ciklusu rezonovanja tipičnom za sisteme bazirane na znanju.

Tok izvršavanja preporuke poteza:

1. korisnik unosi trenutno stanje partije
2. ulazni podaci transformišu se u skup činjenica i ubacuju u radnu memoriju
3. CEP mehanizam analizira sekvencu događaja i generiše izvedene činjenice
4. pokreće se *forward chaining* zaključivanje koje aktivira heuristička pravila
5. *accumulate* operacije agregiraju doprinos pojedinačnih pravila i izračunavaju prioritete akcija
6. *backward chaining* proverava validnost izabrane preporuke
7. sistem vraća rangiranu i objašnjenu odluku korisniku.

Na ovaj način sistem ne donosi odluku direktnim izračunavanjem, već kroz proces postepenog zaključivanja nad bazom znanja.

3. Primer rezonovanja (korak po korak)

Sledeći primer ilustruje način na koji sistem dolazi do preporuke poteza.

Scenario

Partija igre je u sledećem stanju:

- runda 3, preostala su 3 poteza;
- aktivni cilj runde: **najviše ptica u šumi;**
- igrač ima dve ptice u šumi, vodeći protivnik ima tri;
- šuma već omogućava stabilno dobijanje hrane;
- igrač poseduje:
 - jedno jaje,
 - jednu pšenicu i
 - jednog crva;
- u ruci igrača se nalazi ptica vredna 6 poena koja može da se odigra u šumi.

Korak 1 - Ubacivanje činjenica

Ulaz se prevodi u činjenice radne memorije:

```
round = 3
turns_remaining = 3
round_goal = forest_birds
```

```
forest_bird_count = 2
leader_forest_bird_count = 3
```

```
food(wheat) = 1
food(worm) = 1
eggs_available = 1
```

```
bird_in_hand(forest_bird)
bird_cost(forest_bird, wheat + worm)
bird_points(forest_bird, 6)
```

Korak 2 - Nivo 1: osnovni zaključci (*Forward Chaining*)

Aktiviraju se pravila prikladnih akcija:

- ptica je može da se odigra (resursi postoje);
- zahtev za jajima je zadovoljen;
- akcija *PLAY BIRD* postaje validan kandidat.

Novi zaključak:

`playable(forest_bird)`

Korak 3 - Nivo 2: strateški kontekst

Sistem zatim izvodi apstraktne strateške činjenice:

- cilj runde može da stigne da se ispuni (potrebna još jedna ptica u šumi);
- broj poteza je mali: detektuje se *round_goal_urgency*;
- šuma već sadrži bar dve ptice: zaključuje se razvijeno stanište za proizvodnju hrane.

Novi zaključci:

`round_goal_reachable`
`round_goal_urgent`
`developed_food_habitat`

Ovde se vidi prelazak sa mehanike igre na strateško rezonovanje.

Korak 4 - Nivo 3: akumulacija prioriteta (*Accumulate*)

Pravila dodeljuju parcijalne doprinose akcijama.

PLAY BIRD

- +3 dostupnost resursa
- +4 napredak ka cilju runde
- +2 jačanje postojećeg *engine*-a

Ukupno: 9

GAIN FOOD

- +3 stabilnost resursa

LAY EGGS

- +2 (jaja nisu trenutno neophodna, mogu i ne moraju da se polože)

DRAW CARDS

- +1 (ruka nije prazna)

Accumulate operator sabira doprinose i formira rangiranje akcija.

Korak 5 - Generisanje preporuke

Forward chaining zaključuje:

```
recommended_action = PLAY_BIRD(forest_bird)
priority = 9
```

Korak 6 - Validacija (*Backward Chaining*)

Sistem proverava cilj:

„Da li je igranje ptice optimalna akcija?“

Provera uslova:

- ptica može da se odigra ✓
- doprinosi cilju runde ✓
- ne postoji kritičan nedostatak resursa ✓
- poboljšava buduće poteze ✓

Preporuka potvrđena.

Korak 7 - Razumljiv izlaz sistema

Korisnik dobija rezultat:

Preporuka: Igrati pticu X u šumu

Prioritet: 9

Obrazloženje:

- izjednačava igrača sa vodećim igračem u cilju runde,
- trenutno je finansijski izvodljivo,
- dodatno jača produkciju hrane za naredne poteze.

Arhitektura sistema

Predloženi sistem biće realizovan kao višeslojna aplikacija koja razdvaja korisnički interfejs, domensku logiku i mehanizam zaključivanja.

Korisnički interfejs biće razvijen korišćenjem **Angular** radnog okvira i služiće za unos trenutnog stanja partije, prikaz preporučenih akcija i objašnjenja procesa rezonovanja.

Centralna logika sistema biće implementirana u programskom jeziku **Rust** zbog svojih visokih performansi, memorijske bezbednosti i bezbednosti tipova. Rust je pogodan za implementaciju *rule-based* sistema zbog biblioteke [rust-rule-engine](#), koja ima sve neophodne funkcionalnosti potrebne za realizaciju opisanog mehanizma zaključivanja.

Aplikacija će biti realizovana kao desktop i mobilna Android aplikacija. Za povezivanje Angular interfejsa i Rust logike koristiće se radni okvir [Tauri](#), koji omogućava izradu *lightweight cross-platform* aplikacija koji dele isti *codebase*.